Spoken Tutorial – Implementing HNSW with Vector Databases

Title: Implementing HNSW with Vector Databases

Module	Vector Search Technologies
Episode	Episode 5: Hands-on with FAISS
Learning Objective	At the end of this tutorial learner will be able to 1. Compare types of Vector Databases 2. Install and configure the FAISS library 3. Implement HNSW index parameters in Python 4. Perform vector searches using FAISS
Approx. Duration	3-4 min
Outline	• Introduction to Vector Databases • Overview of FAISS, Open-Source, and Managed tools • Hands-on installation of FAISS • Configuring HNSW parameters (d, M, efSearch) • Running a vector search query
Meta Tags	HNSW, FAISS, Vector Database, Python, Search Algorithms, Chroma, Pinecone
Pre-requisite Tutorial	Basic knowledge of Python programming and the HNSW algorithm concepts.

Script

Visual Cue	Narration
Title text on a clean background with vector network illustrations	Welcome to this Spoken Tutorial on Implementing HNSW with Vector Databases. In this session, we will move from theory to practice.
List of learning objectives	In this tutorial, we will learn how to compare different Vector Database tools. We will install the FAISS library. We will configure HNSW parameters like M and efSearch. Finally, we will perform vector searches in Python.
Icons of Python, Jupyter, and operating systems	To record this tutorial, I am using a standard Python environment. Ensure you have Python 3.8 or higher installed. I recommend using Jupyter Notebook for this exercise. You will need an internet connection to install packages.

Diagram showing a query entering a database and highlighting a specific vector cluster	We store our meaning coordinates, or vectors, in a Vector Database. Think of this as a highly optimized address book for vectors. When a new query comes in, the database converts it into a vector. It then uses the internal HNSW algorithm to instantly find the closest matches.
Three columns comparing FAISS, Open Source DBs, and Managed Services	You have three main choices for your library tool. First, In-Memory Libraries like FAISS from Meta. Second, Open-Source Databases like Chroma, Weaviate, or Qdrant. Third, Managed Services like Pinecone.
Comparison chart showing pros and cons	FAISS is blazing fast and runs in your Python script. It is not a full database. Open-Source databases handle billions of vectors. They require server maintenance. Managed services are easy to use. They cost money and hold your data remotely. We will start with FAISS to learn the core parameters.
Screenshot of terminal executing pip install faiss-cpu numpy	Let's begin the hands-on practice. Step 1: Open your terminal or command prompt. Step 2: Type pip install faiss-cpu numpy. Then press Enter. Step 3: Wait for the installation to complete. We use the CPU version for simplicity in this demo.
Screenshot of Jupyter notebook with import statements	Now, let's write the code. Step 1: Open your Python editor or Jupyter Notebook. Step 2: Type import numpy as np. Step 3: On a new line, type import faiss. Step 4: Run the cell to verify imports work.
Code snippet defining d=64 and creating random numpy array	We need some data to work with. Step 1: Type d = 64 to set the vector dimension. Step 2: Create a random dataset. Type xb = np.random.random((1000, d)).astype('float32'). Step 3: Press Enter to run. FAISS requires float32 data types.
Code snippet showing index initialization with M parameter	Now we create the HNSW index. Step 1: Set the number of neighbors. Type M = 32. This M parameter controls the connections per node. Step 2: Initialize the index. Type index = faiss.IndexHNSWFlat(d, M). Step 3: Run this line to build the empty structure.

Code snippet adding data and printing total count	Let's populate our address book. Step 1: Type index.add(xb) to add our random vectors. Step 2: Check if it worked by typing print(index.ntotal). Step 3: Run the code. You should see the number 1000 printed on the screen.
Code snippet setting efSearch parameter	We can tune the search accuracy. Step 1: Type index.hnsw.efSearch = 64. This parameter controls how deep we search the graph. A higher number is more accurate but slower. A lower number is faster but less accurate.
Code snippet performing search and printing indices	Finally, let's search for the nearest neighbors. Step 1: Create a query vector. Type xq = np.random.random((1, d)).astype('float32'). Step 2: Decide how many results we want. Type k = 4. Step 3: Search by typing D, I = index.search(xq, k). Step 4: Print the results with print(I).
Summary points bulleted list	Let's summarize what we learned. FAISS is a powerful tool for research and prototyping. We configured d for dimensions. We configured M for graph connections. We also adjusted efSearch to balance speed and accuracy.
Assignment text asking to change M value	Here is your assignment. Create a new index with a different M value. For example, use M = 4. Perform the same search with the same query vector. Compare the results with the previous index. Observe how the connectivity affects the search output.
Logos of EduPyramids and IIT Bombay	This Spoken Tutorial is brought to you by EduPyramids Educational Services Private Limited. It is also brought to you by SINE, IIT Bombay. Thank you for joining!